What is Package in Java?

A package is nothing but a physical folder structure (directory) that contains a group of related classes, interfaces, and sub-packages according to their functionality. It provides a convenient way to organize your work. The Java language has various in-built packages.

For example, java.lang, java.util, java.io, and java.net. All these packages are defined as a very clear and systematic packaging mechanism for categorizing and managing. Let's understand it with the help of real-time examples.

Realtime Example of Packages in Java

A real-life example is when you download a movie, song, or game, you make a different folder for each category, like movie, song, etc. In the same way, a group of packages in Java is just like a library.

The classes and interfaces of a package are like books in the library that can reuse several times when we need them. This reusability nature of packages makes programming easy.

Therefore, when you create any software or application in Java programming language, they contain hundreds or thousands of individual classes and interfaces. So, they must be organized into a meaningful package name to make proper sense, and reusing these packages in other programs could be easier.

Advantage of using Packages in Java

1. **Maintenance:** Packages in Java are used for proper maintenance. If any developer newly joined a company, he can easily reach to files needed.

2. **Reusability:** We can place the common code in a common folder so that everybody can check that folder and use it whenever needed.

3. **Name conflict:** Packages help to resolve the naming conflict between the two classes with the same name. Assume that there are two classes with the same name Student.java. Each class will be stored in its own packages, such as stdPack1 and stdPack2 without having any conflict of names.

4. **Organized:** It also helps in organizing the files within our project.

5. **Access Protection:** A package provides access protection. It can be used to provide visibility control. The members of the class can be defined in such a manner that they will be visible only to elements of that package.

Types of Packages in Java

There are mainly two types of packages available in Java. They are:

- User-defined package
- Built-in package (also called predefined package)

User-defined Package in Java

The package which is defined by the user is called user-defined or custom package in Java. It contains user-defined classes and interfaces. Let's understand how to create a user-defined package.

Creating User-defined Package

Java supports a keyword called "package" which is used to create user-defined packages in Java programming. The general syntax to create a package in Java is as:

```
JAVA
package packageName;
```

Here, packageName is the name of a package. The package statement must be the first line in a Java source code file, followed by one or more classes. For example:

```
JAVA
package myPackage;
public class A {
      // class body
}
```

In this example, myPackage is the name of a package. The statement "package myPackage;" signifies that the class "A" belongs to the "myPackage" package.

Naming Convention for User-defined Package in Realtime Project

While developing your project, you must follow some naming conventions regarding packages declaration. Let's take an example to understand the convention.

Look at the below a complete package structure of the project.

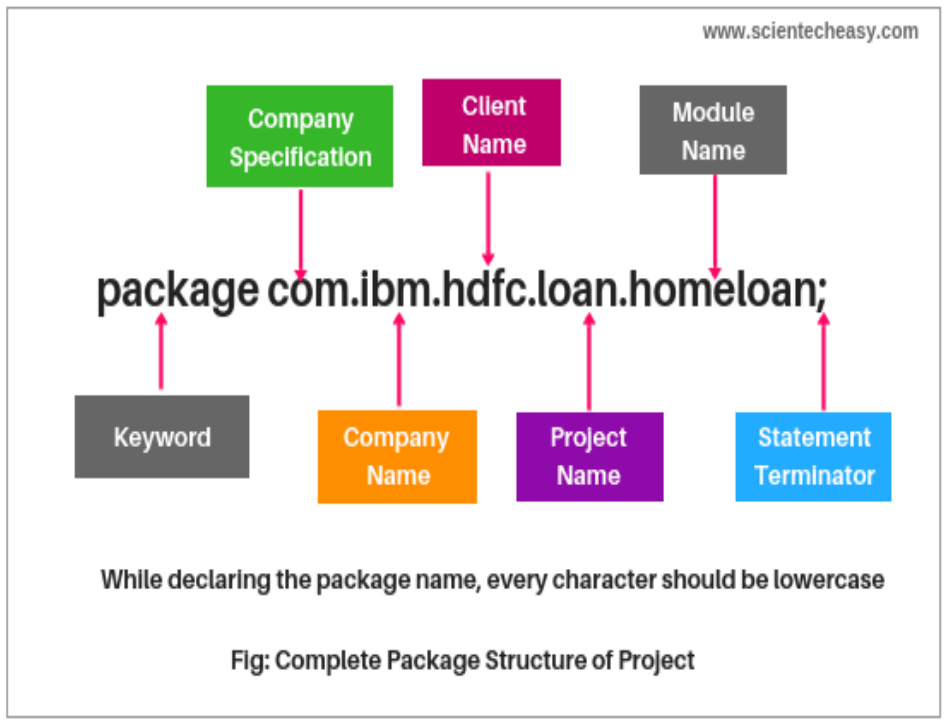


Fig: Complete Package Structure of Project

1. Suppose you are working in IBM and the domain name of IBM is www.ibm.com. You can declare the package by reversing the domain like this:

```
JAVA
package com.ibm;
```

where,

- com → It is generally the company specification name, and the folder starts with com, which is called root folder.
- ibm → Company name where the product is developed. It is the sub folder.

2. hdfc → Client name for which we are developing our product or working on the project.

3. loan → Name of the project.

4. homeloan → It is the name of the modules of the loan project. There are a number of modules in the loan project like a home loan, car loan, or personal loan. Suppose you are working for the Home loan module.

This is a complete packages structure, like a professional which is adopted in the company. Look at another example below:

```
JAVA
package com.tcs.icici.loan.carloan.penalty;
```

Note: Keep in mind Root folder should be always the same for all the classes.

Predefined Packages in Java (Built-in Packages)

Predefined packages in Java are those which are developed by the Sun Microsystem. They are also called built-in packages. These packages consist of a large number of predefined classes, interfaces, and methods that are used by the programmer to perform any task in his programs.

Java APIs contains the following predefined packages, as shown in the below figure:

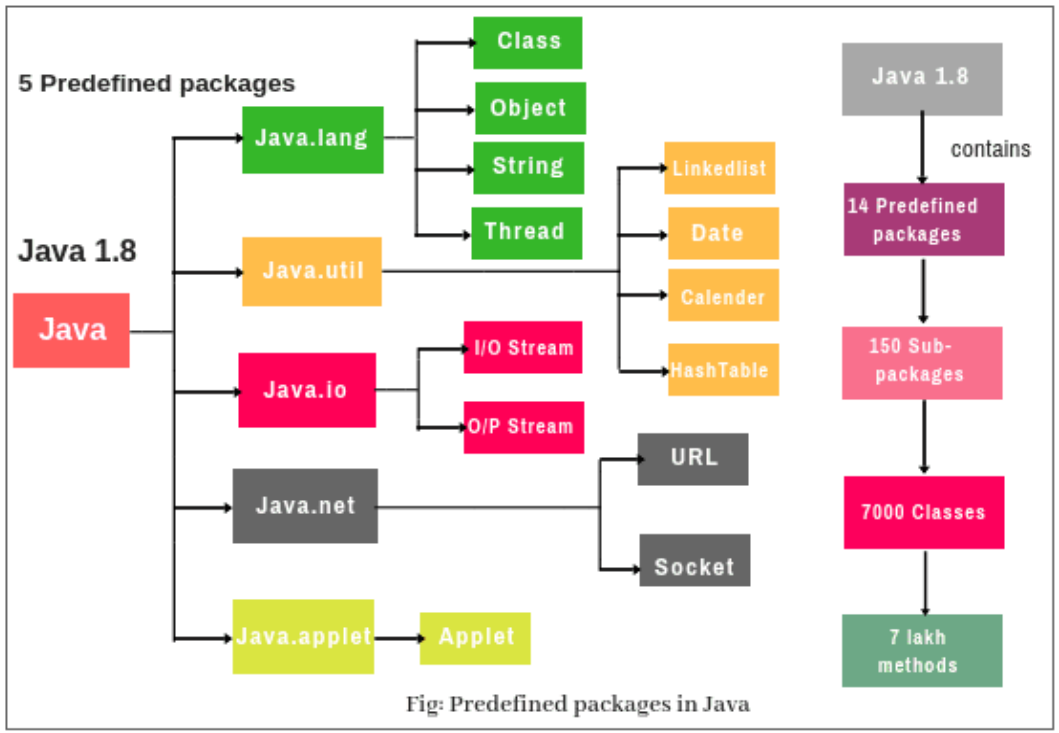


Fig: Predefined packages in Java

Java Core Packages:

1. **Java.lang:** The 'lang' stands for language. The Java language package consists of Java classes and interfaces that form the core of the Java language and the JVM. It is a fundamental package that is useful for writing and executing all Java programs. Examples are classes, objects, string, thread, predefined data types, etc. It is imported automatically into the Java programs.

2. **Java.io:** The 'io' stands for input and output. It provides a set of I/O streams that are used to read and write data to files. A stream represents a flow of data from one place to another place.

3. **Java util:** The 'util' stands for utility. It contains a collection of useful utility classes and related interfaces that implement data structures like LinkedList, Dictionary, HashTable, stack, vector, calender, data utility, etc.

4. **Java.net:** The 'net' stands for network. It contains networking classes and interfaces for networking operations. The programming related to the client-server can be done by using this package.

Window Toolkit and Applet:

1. **Java.awt:** The 'awt' stands for abstract window toolkit. The Abstract window toolkit package contains GUI (Graphical User Interface) elements, such as buttons, lists, menus, and text areas. Programmers can develop programs with colorful screens, paintings, and images, etc using this package.

2. **Java.awt.image:** It contains classes and interfaces for creating images and colors.

3. **Java.applet:** It is used for creating applets. Applets are programs that are executed from the server into the client machine on a network.

4. **Java.text:** This package contains two important classes, such as DateFormat and NumberFormat. The class DateFormat is used to format dates and times. The NumberFormat is used to format numeric values.

5. **Java.sql:** SQL stands for the structured query language. This package is used in a Java program to connect databases like Oracle or Sybase and retrieve the data from them.

Java Package Development from Java 8 Onwards

1. Java predefined supports a group of packages that contains a group of classes and interfaces. These classes and interfaces consist of a group of methods.

For example, Java language contains a package called java.lang which contains string class, StringBuffer class, StringBuilder class, all wrapper classes, runnable interface, etc. String class contains a number of methods such as length(), toUpperCase(), toLowerCase() etc.

2. Java contains 14 main predefined packages. These 14 predefined packages contain nearly 150 sub-packages that consist of a minimum of 7 thousand classes. These 7 thousand classes contain approx 7 lakhs methods.

3. Up to Java 1.7 version contains 13 predefined packages. From Java 1.8 version onwards, one new package is introduced called java.time.

4. Java 9 introduced several new packages, such as:

- java.lang.module
- java.util.spi, jdk.jshell
- java.util.concurrent.Flow
- java.lang.invoke.VarHandle
- jdk.incubator.httpclient.

5. Java 10 introduced relatively few changes compared to Java 9 and did not include any major new packages.

6. Java 11 introduced java.net.http that provides a new HTTP client that supports HTTP/2 and WebSocket.

7. Java 12 and 13 versions did not include any packages.

8. Java 14 had introduced a new package named jdk.jfr.consumer.

9. Java 15 and onwards version did not introduce any new packages.

How to See List of Predefined Packages in Java?

Follow the following steps to see the list of predefined packages in Java.

1. Go to programs files and open them.

2. Now go to Java folder and open it. You will see two folders such as JDK and JRE.

3. Go to JDK folder, extract the src folder. After extracting it, go to Java folder. Here, you will see 14 predefined packages folders such as applet, awt, beans, io, lang, math, net, nio, rmi, security, sql, text, time, and util.

4. Now you open lang package and scroll down. You can see classes like String, StringBuffer, StringBuilder, Thread, etc.

Example 1:

JAVA

```java
// Save as Example.java

// Step 1: Declare package name by reversing domain name, project name 'java', and module name is core java.
   package com.scientecheasy.java.corejava;

// Step 2: Declare class name.
   public class Example
   {
     public static void main(String[] args)
     {
        System.out.println("How to create a Java package");
     }
   }
```

How to Compile Package in Java?

If you are not using any Eclipse IDE, you follow the syntax given below:

```
JAVA
javac -d directory javafilename // syntax to compile the application
```

In the above syntax,

1. javac means *Java compiler*.

2. -d means directory. It creates the folder structure.

3. .(dot) means the current directory. It places the folder structure in the current working directory. For example:

```
JAVA
javac -d.Example.java // Here, Example.java is the file name.
```

So in this way, you must compile application if the application contains a package statement. After the compilation, you can see the folder structure in your system like this:

```
com
  |---> scientecheasy
     |------> java
        |------> corejava
           |------> Example.class
```

How to Run Java Package Program?

You have to use the fully qualified name to execute Java code. The fully qualified name means class name with a complete package structure. Use the below syntax to run Java code.

Syntax:

```
JAVA
java completePackageName.className
```

Now run the above Java code. To Run:

```
JAVA
java com.scientecheasy.java.corejava.Example
```

```
Output:
       How to create a Java package
```

How to Import Package in Java

There are three approaches to import one package into another package in Java.

1. import package.*;
2. import package.classname;
3. Using fully qualified name.

Using package.*

An import is a keyword that is used to make the classes and interfaces of other packages accessible to the current package. If we use package.*, all the classes and interfaces of this package can be accessed (imported) from outside the packages. Let's understand it by a simple example program.

```
// Create a package.
package com.scientecheasy.calculate;

// Create a class with a public access modifier.
// If you use a default access modifier, it cannot be accessible due to default, which is accessible within the same package.
public class Sum
{
// Declare instance variables.
   int a = 20;
   int b = 30;

// Declare method.
   public void cal()
   {
       int s = a + b;
       System.out.println("Sum: " +s);
   }
}
// Create another package.
package com.maths.calculator;

// Importing the entire package into the current package.
import com.scientecheasy.calculate.*;

class SumTest
{
 public static void main(String[] args)
 {
// Create an object of class and call the method using reference variable s.
     Sum s = new Sum();
      s.cal();
   }
}
```

```
Output:
        Sum: 50
```

Using packageName.className

Suppose scientecheasy has information about the Dhanbad city and TCS needs this information. We will declare two modules: Dhanbad and TCS. TCS is using Dhanbad class, but both have different package names. Whenever you are using a class of another package, you must import the package first of all.

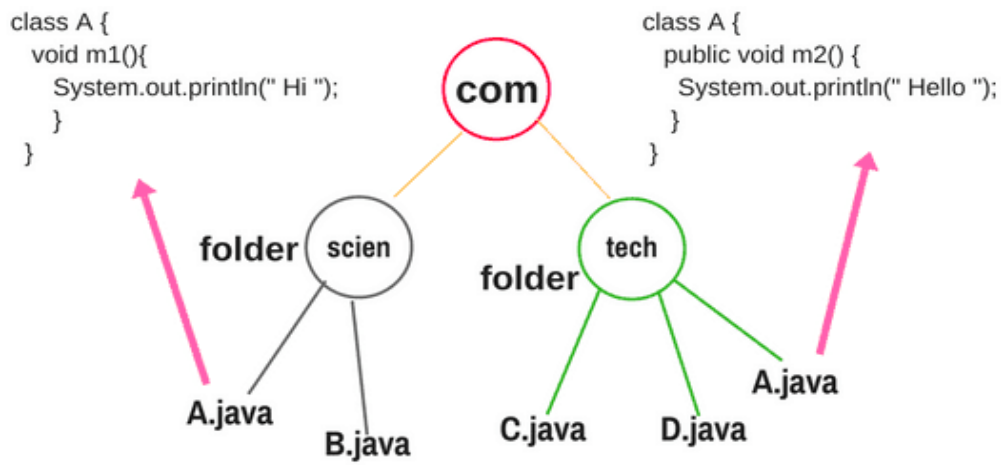
```
// Declare complete package statement.
package com.scientecheasy.state.cityinfo;
public class Dhanbad
{
  public void stateInfo()
  {
    System.out.println("Dhanbad is one of the major cities of Jharkhand");
  }
  public void cityInfo()
  {
    System.out.println("Dhanbad is the coal capital of India.");
  }
}
// Declare complete package statement for TCS.
   package com.tcs.state.requiredinfo;

// Import the package with class name.
    import com.scientecheasy.state.cityinfo.dhanbad;
    class Tcs
    {
     public static void main(String[] args)
     {
       Dhanbad d = new Dhanbad();
        d.stateinfo();
        d.cityinfo();
     }
}
```

```
Output:
            Dhanbad is the first major city of Jharkhand.
            Dhanbad city is called coal capital city of India.
```

Using the fully qualified name

If you use the fully qualified name, there is no need to use an import statement, but in this case, only the declared class of this package can be accessible. It is generally used when two packages have the same class name.

Let's take a scenario to understand the above concept. Consider the below figure.



My requirement to call m1 of class A of sub-package scien and m2 of class A of sub-package tech form class B of sub-package scien.

In the com package, there are two sub-packages "scien" and "tech". The sub-package "scien" contains two class files A.java and B.java. Whereas the sub-package tech contains three class files: C.java, D.java, and A.java.

```
package com.scien;
import com.tech.A;
class B
{
   void m3()
   {
      System.out.println("Hello Java");
   }
public static void main(String[] args)
 {
   A a = new A();
   a.m1();

   A a1 = new A();
    a1.m2;

   B b = new B();
    b.m3();
  }
}
```

Will the above code compile?

1. No: because the statement A a = new A(); does not say anything about class A from which sub-packages (scien or tech) it is referring.

2. No: because the statement A a1 = new A(); is also not saying anything about class A of which sub-packages (scien or tech) it is referring.

3. No: because a.m1() and a1.m2() will get confused to call the method of which package's class. Here, the compiler will be also confused.

In this case, the import is not working. So, we remove the import statement and use the fully qualified name.

```java
package com.scien;
class B
{
  void m3()
  {
     System.out.println("Hello Java");
  }
public static void main(String[] args)
{
   A a = new A(); // keep as it is because it is from the same package "scien".
    a.m1();

   com.tech.A a1 = new com.tech.A(); // It will direct go to tech package and call the method m2.
    a1.m2;

   B b = new B();
    b.m3();
  }
}
```

```
Output:
       Hi
       Hello
       Hello Java
```

Suppose you are not using public with m2() method in the above program, then it will give error " The method m2() from the type A is not visible" because it is a default and default access modifier cannot be accessed from outside the package.